

Proyecto 1 — Procesamiento y Análisis de Datos Masivos

MovieLens 20M · PySpark · MinHash + LSH

Curso: Data Mining · UTEC · Prof. Heider Sanchez · Autor: Johar · Abril 2026

Resumen

Este reporte sintetiza el procesamiento y análisis del dataset MovieLens 20M (20,000,263 ratings, 138,493 usuarios, 27,278 películas) usando Apache Spark sobre un clúster Docker. Se ejecutaron tres fases: (i) preprocesamiento y EDA distribuidos, (ii) procesamiento MapReduce con benchmark RDD vs DataFrame y top-20 de películas, y (iii) búsqueda de pares similares con Jaccard exacto, MinHashing y LSH bajo distintas configuraciones de bandas. El sparsity de 99.46% justifica el uso de técnicas aproximadas; MinHash logra correlación de Pearson 0.89 con Jaccard exacto y la configuración LSH óptima ($b=25$, $r=4$) alcanza precisión 0.85.

1. Metodología

1.1 Entorno de ejecución

Apache Spark 4.1.1 corriendo sobre Docker Compose (1 master + 2 workers, 20 cores totales, 13.3 GiB). Los notebooks Python interactúan con el clúster vía PySpark sobre Mac M1 Pro. La SparkSession local se usa solo para verificación; el trabajo principal corre en modo distribuido.

1.2 Parte I — Preprocesamiento y EDA

Se cargaron los CSV con esquema explícito y se inspeccionaron tipos, conteos y rangos. No se hallaron nulos, duplicados ni ratings fuera de la escala [0.5, 5.0]. Las 246 películas sin género se etiquetaron como Unknown y los 2,334 tags técnicos bd-r se eliminaron. Se unificaron los géneros (Sci-Fi → Science Fiction; Film-Noir → Film Noir; Children → Children's) y se descartó IMAX por ser un formato. La binarización like/dislike usa el umbral 3.5, justificado por la media del dataset (3.53). Los datasets limpios se persistieron en Parquet para reuso en fases posteriores.

1.3 Parte II — Procesamiento distribuido

Se replicó el conteo de ratings por película con dos APIs: DataFrame (groupBy + count + orderBy) y RDD estilo MapReduce (map + reduceByKey + sortBy). Se midieron tres operaciones (conteo por película, top-20 con join a movies y conteo por usuario) registrando wall-clock time para comparar Catalyst-optimizado contra ejecución manual de RDD.

1.4 Parte III — Similitud, MinHash y LSH

Cada película se representa como el conjunto de usuarios que le dieron like (Opción A). Se descartan películas con menos de 50 likes (justificado por análisis de sensibilidad: con ≥ 50 elementos el cambio del Jaccard al añadir un usuario cae por debajo del 1.5%), quedando 8,362 películas. Se calcula el Jaccard exacto sobre una muestra de 200 películas (19,900 pares) como ground truth. Las firmas MinHash se construyen con 100 funciones hash de la forma $h(x) = (a \cdot x + b) \bmod p$ mod N ($p = 2,147,483,647$). Se compara el Jaccard estimado contra el exacto (MAE, correlación de Pearson) y se evalúa LSH variando b y r con $b \cdot r = 100$ constante; se contabilizan TP, FP, FN para precisión y recall.

2. Hallazgos del EDA

2.1 Distribución de ratings y sparsity

Los usuarios califican en su mayoría positivamente: la media es 3.53 y la mediana 3.5. Los ratings enteros (3, 4, 5) son notoriamente más frecuentes que los medios (3.5, 4.5), evidenciando preferencia por números redondos. El 3.5 se eligió como umbral de binarización porque es el valor donde el usuario conscientemente da media estrella extra. Resultado: 61% likes, 39% dislikes.

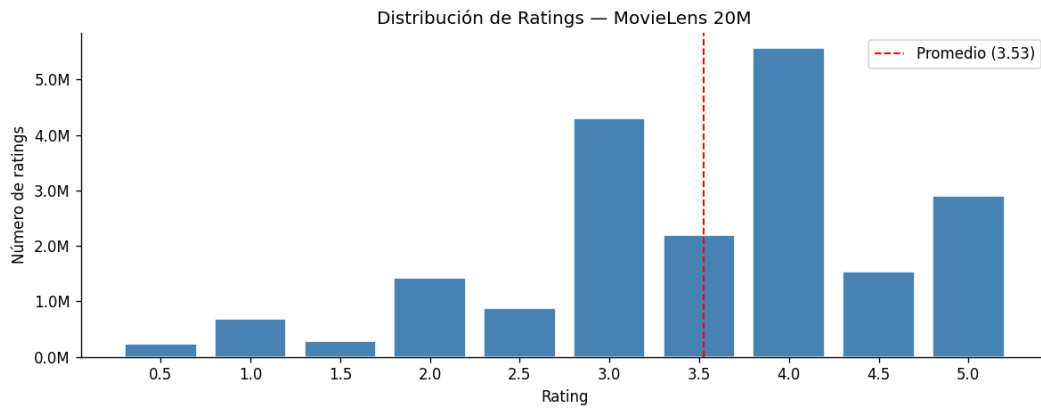


Figura 1. Distribución de los 20M ratings — sesgo hacia ratings altos y preferencia por enteros.

La matriz usuario-película tiene $138,493 \times 26,744 \approx 3.7 \times 10^9$ celdas, pero solo 20M están llenas. El 99.46% está vacío. Esta densidad bajísima justifica el uso de MinHash y LSH: comparar pares exactos sería $O(N^2)$ y la mayoría de comparaciones daría intersección nula.

Sparsity de la matriz usuario-película

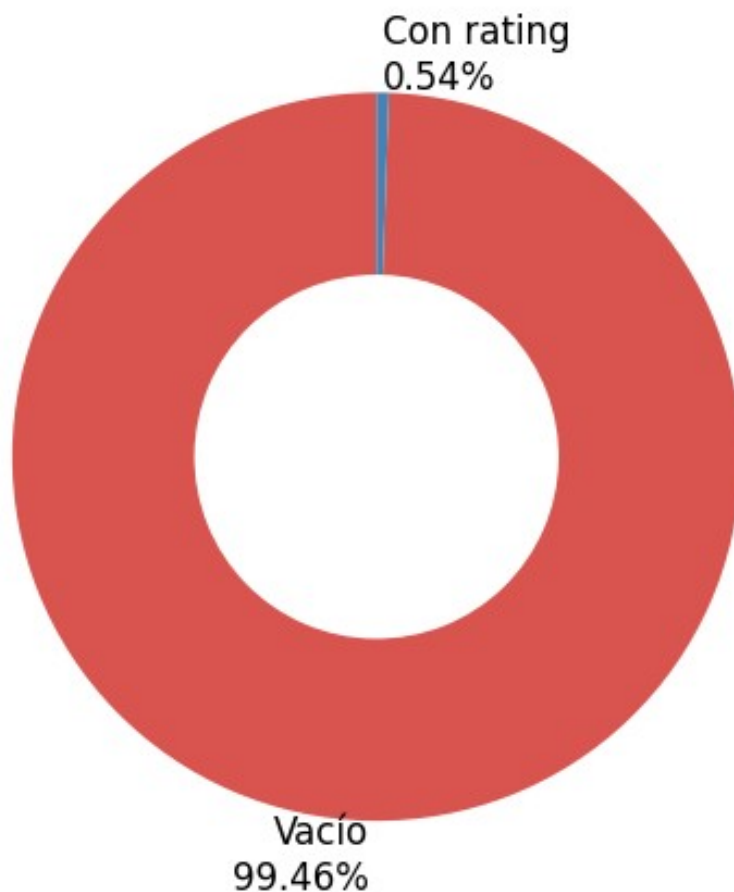


Figura 2. Heatmap de la submatriz usuario-película más densa (99.46% sparsity total).

2.2 Actividad por usuario y por película — power law

Tanto la actividad de usuarios como la popularidad de películas siguen una distribución de cola larga. El 75% de los usuarios tiene entre 20 y 155 ratings, pero algunos outliers superan los 9,000 ratings. Para películas, $P50 = 18$ (la mitad tiene 18 o menos ratings: cine de nicho), $P75 = 205$, y la más calificada es Pulp Fiction con 67,310 ratings. El top 25% concentra casi todos los ratings del dataset.

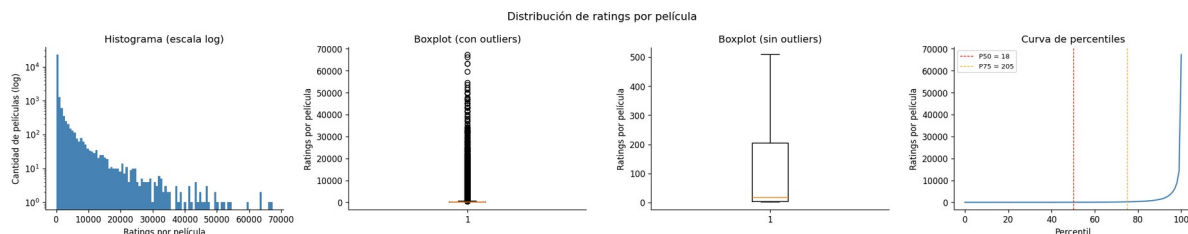


Figura 3. Distribución de ratings por película — long tail clásico.

2.3 Géneros y evolución temporal

Drama (13,344) y Comedy (8,374) dominan más del 50% del catálogo; Western (676) y Film-Noir (330) son nichos. Como una película puede tener varios géneros, los conteos no suman al total. Temporalmente, la actividad muestra picos en 2000 y 2005 — alineados con el auge del DVD y la consolidación de plataformas de recomendación online — y una caída posterior cuando los usuarios migraron a Netflix y YouTube. La caída de 2015 es artificial: el dataset se exportó a mitad de año.

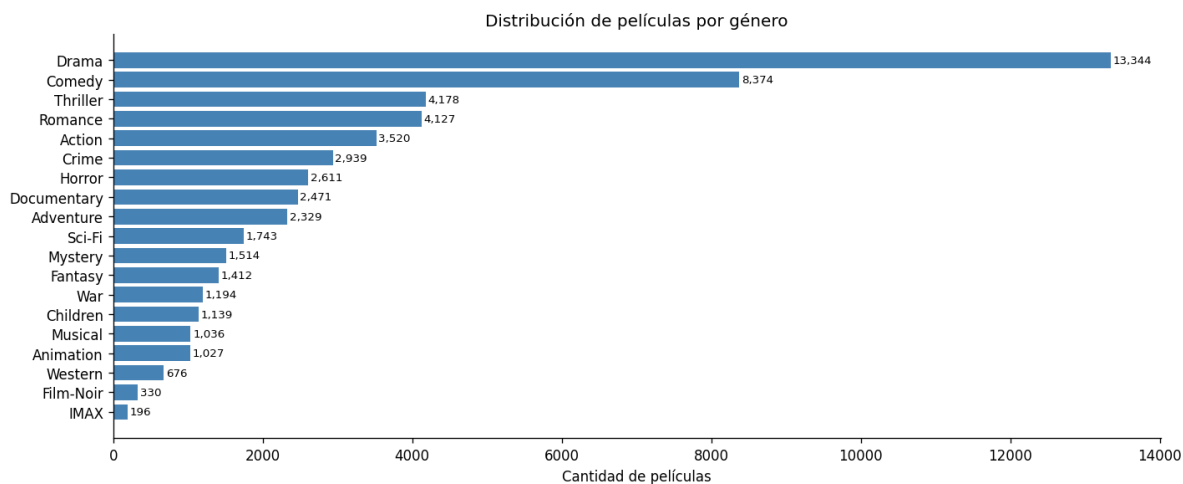


Figura 4. Distribución de géneros — Drama y Comedy concentran más del 50% del catálogo.

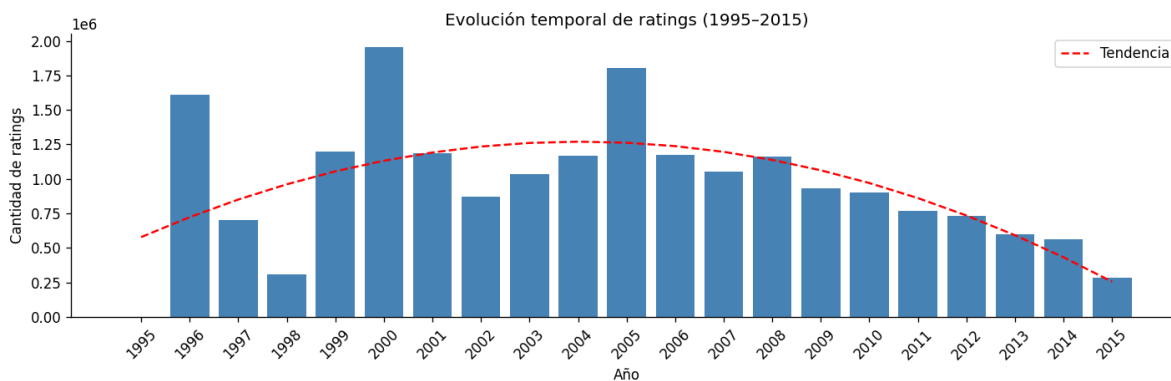


Figura 5. Evolución temporal de ratings — picos en 2000 y 2005.

2.4 Tags de usuario

Los tags más usados son sci-fi (3,576), atmospheric, surreal y comedy. Coexisten dos tipos: tags de género (que duplican movies.csv) y descriptores de estilo o experiencia. Esta señal alternativa se consideró para representar películas en MinHash, pero se descartó por menor cobertura: solo el 3% de las películas tiene tags suficientes.

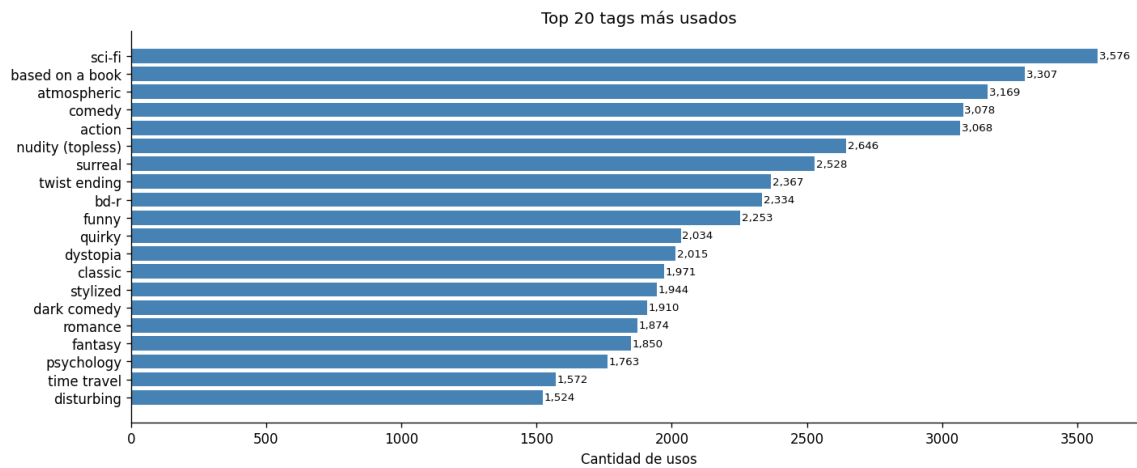


Figura 6. Top tags de usuario — sci-fi domina con 3,576 usos.

3. Resultados

3.1 Top-20 películas más calificadas (Parte II)

El ranking refleja el cine icónico de los 90s: Pulp Fiction (1994) lidera con 67,310 ratings, seguido por Forrest Gump y The Shawshank Redemption. Se obtuvo con `DataFrame.groupby("movieId").count()` y join a `movies_clean` para los títulos.

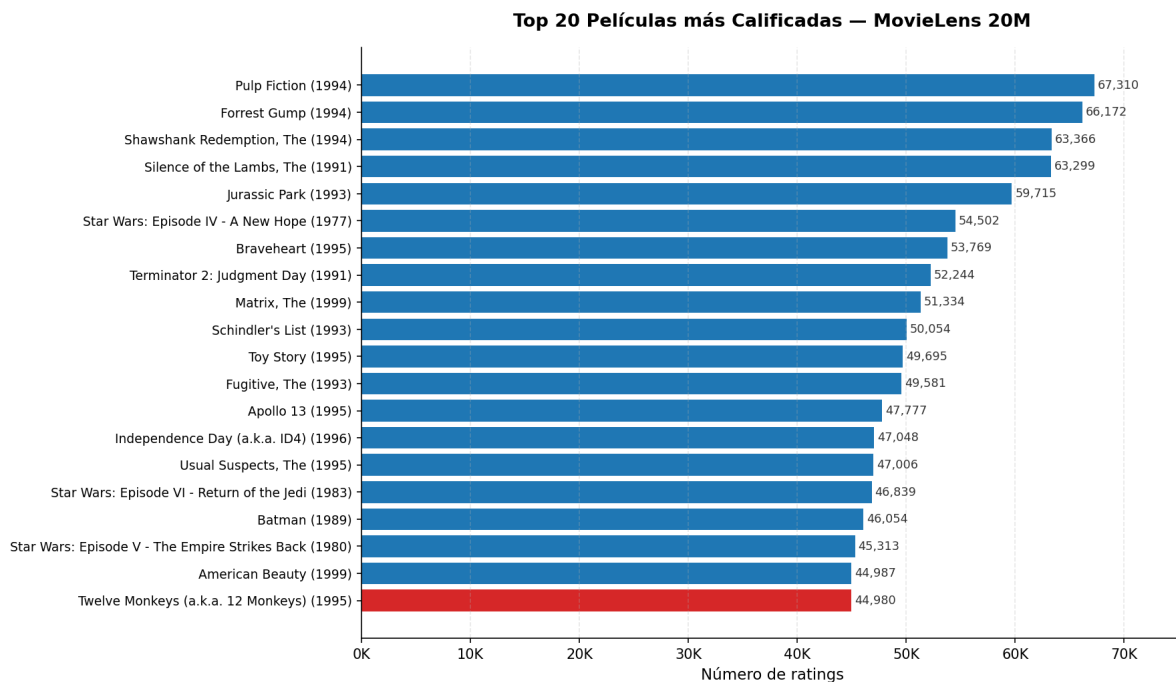


Figura 7. Top-20 películas más calificadas en MovieLens 20M.

#	Película	# ratings
1	Pulp Fiction (1994)	67,310
2	Forrest Gump (1994)	66,172
3	The Shawshank Redemption (1994)	63,366
4	The Silence of the Lambs (1991)	63,299
5	Jurassic Park (1993)	59,715
6	Star Wars Episode IV (1977)	54,502
7	Braveheart (1995)	53,769
8	Terminator 2 (1991)	52,244
9	The Matrix (1999)	51,334
10	Schindler's List (1993)	50,054

--	--	--

3.2 RDD vs DataFrame

DataFrame es consistentemente más rápido que RDD gracias al optimizador Catalyst y al motor Tungsten que genera código JVM optimizado. La ventaja crece con la complejidad de la operación.

Operación	Speedup DataFrame
Conteo de ratings por película	~119× (con caché)
Top-20 con join a movies	~10×
Conteo de ratings por usuario	~14×

El 119× del primer caso está inflado porque DataFrame aprovechó datos cacheados; los 10×-14× son representativos de un escenario sin caché. La conclusión práctica: para operaciones SQL-style en producción, DataFrame API es la opción por defecto. RDD sigue siendo útil cuando se requiere control fino sobre la partición o lógica que no se expresa con SQL.

3.3 Jaccard exacto vs MinHash (Parte III)

Sobre 19,900 pares, MinHash logra un MAE de 0.0104 y correlación de Pearson de 0.8894 ($p < 1e-300$). El error máximo individual es 0.147, concentrado en pares de baja similitud (un par con $J = 0.07$ puede pasar a estimación 0.18 por azar). En el rango de pares relevantes ($J \geq 0.20$) la correlación es prácticamente lineal, por lo que MinHash preserva el ranking — que es lo que importa para un recomendador.

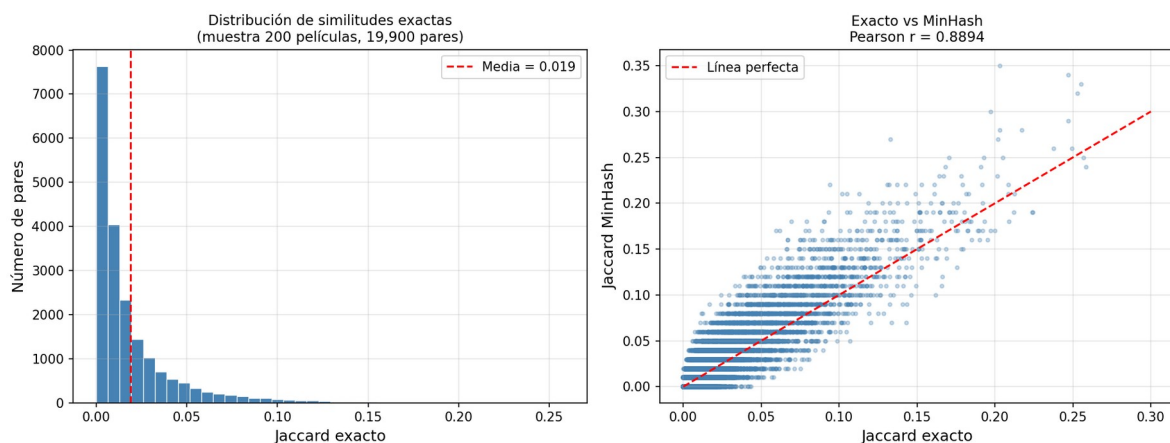


Figura 8. Jaccard exacto vs estimado por MinHash sobre 19,900 pares.

3.4 LSH: precisión, recall y trade-off

Variando b y r con $b \cdot r = 100$ constante, se observa el trade-off clásico de LSH: muchas bandas (b grande, $r=1$) maximizan recall pero la precisión colapsa por exceso de candidatos. Pocas bandas ($b=10$, $r=10$) son demasiado estrictas y ningún par supera el filtro. La configuración $b=25$, $r=4$ ofrece el mejor balance.

b	r	Candidatos	TP	FP	FN	Precisión	Recall
100	1	13,090	404	12,686	0	0.031	1.000
50	2	1,101	268	833	136	0.243	0.663
25	4	20	17	3	387	0.850	0.042
20	5	1	0	1	404	0.000	0.000

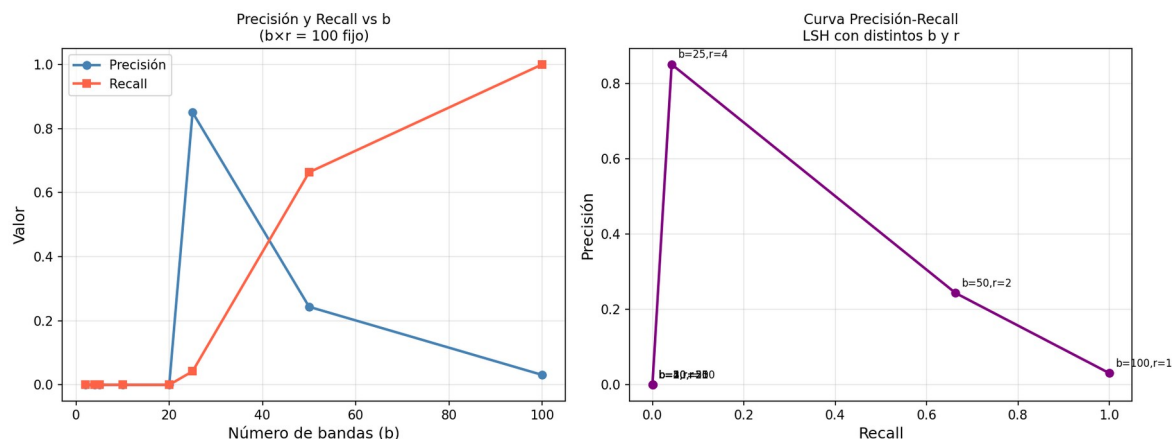


Figura 9. Curva precisión-recall de LSH variando bandas.

MLlib MinHashLSH con threshold 0.20 detectó 22 pares; los 17 verdaderos positivos del banding manual están contenidos en ese resultado. MLlib es eficiente con thresholds altos (Jaccard ≥ 0.5) pero menos controlable cuando se quiere ajustar finamente el balance precisión-recall, donde el banding manual gana flexibilidad.

4. Discusión crítica

Exactitud vs eficiencia. Comparar todas las películas con Jaccard exacto cuesta $O(N^2) \approx 365$ millones de pares para 27K películas. MinHash + LSH reducen este costo a $O(N)$ candidatos a costa de un error controlado. En este experimento la pérdida de información del MinHash es mínima (correlación 0.89 con el exacto), lo que confirma su viabilidad para sistemas de recomendación a gran escala.

Impacto de FP y FN en producción. Un falso positivo en un recomendador es relativamente barato: ofrece una película que el usuario probablemente no consumirá. Un falso negativo es más costoso: deja de recomendar una película que le habría gustado, perdiendo engagement. Por eso la configuración óptima depende del objetivo: para diversidad y descubrimiento conviene b alto (más recall); para listas curadas de alta confianza, b bajo (más precisión).

¿Cuándo usar Spark? Con 20M ratings y un Mac M1 Pro, las operaciones más pesadas (top-20 con join, MinHash sobre 8K películas) corren en minutos en el clúster Docker local. Para datasets de este tamaño, pandas en memoria también funcionaría con cuidado, pero no escala a 200M ratings o más. Spark se justifica cuando: (i) los datos no caben en RAM, (ii) se necesita paralelismo real para operaciones SQL, o (iii) se quiere portabilidad a un clúster productivo. El sobrecosto de configuración es notable y no compensa para datasets pequeños.

Aplicabilidad real. El pipeline Jaccard $\rightarrow$ MinHash $\rightarrow$ LSH es exactamente lo que usa Netflix para encontrar películas similares en su catálogo, Spotify para canciones y Amazon para productos. La representación como conjunto de usuarios con like es robusta porque no requiere features explícitas (géneros, tags), aprovechando solo el comportamiento agregado.

Cuellos de botella observados: (i) el join de ratings (20M filas) con movies_clean genera shuffle costoso; broadcast join sobre movies_clean (27K filas) lo elimina. (ii) Calcular Jaccard exacto sobre toda la muestra requiere mantener los sets en memoria; particionar por hash de movieid ayuda. (iii) La conversión de RDD a DataFrame durante la Fase 3 fue innecesaria en algunos puntos y duplicó el cómputo.

5. Conclusiones

El proyecto demuestra empíricamente tres puntos clave: (1) el preprocesamiento riguroso (limpieza, unificación de géneros, binarización justificada) es prerequisite para cualquier técnica de minería posterior; (2) DataFrame API es $\sim 10\times$ más rápido que RDD en operaciones SQL-style sobre Spark, lo que no significa que RDD sea obsoleto sino que ya no es la primera opción para queries declarativos; (3) MinHash+LSH es una solución eficiente y precisa para encontrar pares similares en datasets malos, con correlación 0.89 frente a Jaccard exacto y precisión de 0.85 en la configuración óptima (b=25, r=4). La Parte IV (FP-Growth para reglas de asociación) queda pendiente.